

small

DEVOIR SURVEILLÉ

Programmation C Avancée – L3 GLSI – Université Cheikh Anta Diop de Dakar

Dr. El Hadji Bassirou TOURÉ – Département de Mathématiques et Informatique

CORRIGÉ

Nom :	Prénom :
Section :	Durée : 1h30 Note : _____ /20

Consignes : Aucun document autorisé. Calculatrice interdite. Répondre directement sur cette feuille.**Machine :** x86-64, little-endian. `sizeof(int)=4`, `sizeof(short)=2`, `sizeof(char)=1`, `sizeof(pointeur)=8`.

Partie A – Fondamentaux C

(5 points)

Exercice 1 : Prédiction de sortie

3 pts

Pour chaque programme, écrire la sortie **exacte** (caractère par caractère).

Programme P1 :

```
#include <stdio.h>
int main() {
    char c = 200;
    unsigned char uc = 200;
    printf("%d %u\n", c, uc);
    int x = -1; unsigned int y = 1;
    printf("%d\n", x < y);
    return 0;
}
```

Sortie P1 : **-56 200****0***char est signé : 200 = 0xC8 → -56 en complément à deux sur 8 bits. Pour x < y : x (-1) est converti en unsigned → 2³² - 1 = 4294967295 > 1 → faux → 0.*

Programme P2 :

```
#include <stdio.h>
int f(void) { static int n = 0; return n++; }
int g(void) { int n = 0; return n++; }
int main() {
    printf("%d %d %d\n", f(), f(), f());
    printf("%d %d %d\n", g(), g(), g());
    return 0;
}
```

Sortie P2, ligne 1 : **2 1 0**Sortie P2, ligne 2 : **0 0 0***L'ordre d'évaluation des arguments est non spécifié (pas UB). f() utilise static : les 3 appels retournent 0, 1, 2. En évaluation droite-à-gauche (x86 GCC typique) : 2 1 0. Accepter toute permutation de {0,1,2}. g() réinitialise n à chaque appel → toujours 0.*

Programme P3 :

```
#include <stdio.h>
int main() {
    int a = 0, b = 3, c = 0;
    if (a++ || b-- && c++)
        printf("V ");
    else
        printf("F ");
    printf("%d %d %d\n", a, b, c);
    return 0;
}
```

Sortie P3 : **F 1 2 1***## a priorité sur ||. a++ → valeur 0 (a devient 1), gauche de || faux → évaluer droite. b-- → valeur 3 (b devient 2), vrai → évaluer droite de ## : c++ → valeur 0 (c devient 1), faux. Résultat : 0 || (3 && 0) = 0 || 0 = 0 → branche else.*

Exercice 2 : Défini ou Undefined Behavior ?

2 pts

Pour chaque expression, cocher la case correspondante. Si l'expression est définie, donner la valeur.

Expression	Défini	UB	Valeur (si déf.)
<code>int a=10; int b=(a++, ++a, a+1);</code>	☒	☐	13
<code>int c = INT_MAX + 1;</code>	☐	☒	
<code>unsigned u = UINT_MAX + 1;</code>	☒	☐	0
<code>int e; printf("%d", e);</code>	☐	☒	
<code>sizeof(x = 999)</code> (x est un int)	☒	☐	4

(1) L'opérateur virgule est un point de séquence : `a++` (`a=11`), `++a` (`a=12`), `a+1=13`. (2) Overflow signé = UB. (3) Overflow unsigned = wrap modulo 2^{32} . (4) Lecture de variable non initialisée = UB. (5) `sizeof` n'évalue pas son opérande, retourne la taille du type = 4.

Partie B – Représentation Numérique**(7 points)****Exercice 3 : Conversions de base****2 pts**Compléter le tableau (valeurs **non signées** sur 8 bits) :

Binaire (8 bits)	Hexadécimal	Décimal
10110100	0xB4	180
11010111	0xD7	215
01100011	0x63	99
01101110	0x6E	110

Exercice 4 : Même motif, deux interprétations**2 pts**

Pour chaque motif binaire sur 8 bits, calculer la valeur en unsigned (B2U) et en signed (B2T).

Rappels : $B2U_8(\vec{x}) = \sum_{i=0}^7 x_i \cdot 2^i$ $B2T_8(\vec{x}) = -x_7 \cdot 2^7 + \sum_{i=0}^6 x_i \cdot 2^i$

Binaire (8 bits)	B2U (unsigned)	B2T (signed)
11010011	211	-45
10000001	129	-127
01100100	100	100

*11010011 : B2U = 128+64+16+2+1 = 211. B2T = -128+64+16+2+1 = -45.**10000001 : B2U = 128+1 = 129. B2T = -128+1 = -127.**01100100 : MSB = 0, donc B2U = B2T = 64+32+4 = 100.***Exercice 5 : Décodage IEEE 754 (float 32 bits)****2 pts**

Le float 0xC2ED0000 est stocké en mémoire. Décoder ce nombre étape par étape.

Étape	Réponse
Écriture en binaire (32 bits)	1 10000101 110110100000000000000000
Signe s (1 bit)	$s = 1$ (négatif)
Exposant brut (8 bits, en décimal)	$10000101_2 = 133$
Exposant réel $E = \text{exp} - 127$	$E = 133 - 127 = 6$
Fraction (23 bits)	110110100000000000000000
Mantisse $M = 1.\text{fraction}$	$M = 1.1101101_2 = 1.8515625$
Valeur $(-1)^s \times M \times 2^E$	$-1 \times 1.8515625 \times 64 = -118.5$

 *$M = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{16} + \frac{1}{32} + \frac{1}{128} = 1.8515625$. $1.8515625 \times 2^6 = 118.5$.***Exercice 6 : Pièges des entiers****1 pt**

Donner le résultat de chaque expression C et justifier brièvement.

Expression	Résultat	Justification
(uint8_t)(250 + 10)	4	$260 \bmod 256 = 4$ (wrap unsigned 8 bits)
-1 < 0U	0 (faux)	-1 converti en unsigned $\rightarrow 2^{32}-1 > 0$
(int8_t)(-100 - 50)	106	$-150 + 256 = 106$ (plage [-128, 127])

Partie C – Organisation Mémoire

(8 points)

Exercice 7 : Endianness

1 pt

La valeur 0xCAFE1337 est stockée à l'adresse 0x40 sur 4 octets.

	0x40	0x41	0x42	0x43
Big-endian	CA	FE	13	37
Little-endian	37	13	FE	CA

Lecture d'un short (2 octets) à l'adresse 0x40 en little-endian : **0x1337**

LE : octets 37, 13 aux adresses 0x40, 0x41 → reconstitution little-endian → 0x1337.

Exercice 8 : Arithmétique des pointeurs

3 pts

```
int tab[6] = {10, 20, 30, 40, 50, 60};    /* &tab[0] = 0x7000 */
int *p = tab + 2;
int *q = &tab[5];
```

Expression	Valeur
p (adresse)	0x7008 <i>(0x7000 + 2 × 4)</i>
*p	30 <i>(= tab[2])</i>
q - p	3 <i>((0x7014 - 0x7008) / 4 = nombre d'éléments)</i>
*(p + 3)	60 <i>(= tab[2+3] = tab[5])</i>
p[-1]	20 <i>(= *(p-1) = tab[1])</i>
(char*)q - (char*)p	12 <i>(0x7014 - 0x7008 = 12 octets)</i>
((short*)0x7000) + 5	0x700A <i>(0x7000 + 5 × 2)</i>

Exercice 9 : Forensics Mémoire

4 pts

Machine x86-64, little-endian. Dump hexadécimal de la mémoire :

Adresse	+0	+1	+2	+3	+4	+5	+6	+7
0xA0	2A	00	00	00	FF	FF	FF	FF
0xA8	A4	00	00	00	00	00	00	00
0xB0	A8	00	00	00	00	00	00	00
0xB8	44	61	6B	61	72	00	00	00
0xC0	B8	00	00	00	00	00	00	00

Table ASCII : D=0x44 a=0x61 k=0x6B r=0x72 NUL=0x00

Déclaration des variables :

```

int    a;          /* 0a = 0xA0 */
int    b;          /* 0b = 0xA4 */
int    *p;         /* 0p = 0xA8 */
int    **pp;       /* 0pp = 0xB0 */
char    s[8];      /* 0s[0] = 0xB8 */
char    *t;        /* 0t = 0xC0 */

```

9a) Valeurs des variables

1 pt

Variable	Valeur
a	42 (0xA0-0xA3 : 2A 00 00 00 → LE → 0x0000002A = 42)
b (en décimal signé <i>et</i> en hex)	-1 / 0xFFFFFFFF (0xA4-0xA7 : FF FF FF FF)
p (adresse pointée)	0xA4 (0xA8-0xAF : 44 00...00 → pointe vers b)
pp (adresse pointée)	0xA8 (0xB0-0xB7 : A8 00...00 → pointe vers p)
t (adresse pointée)	0xB8 (0xC0-0xC7 : B8 00...00 → pointe vers s)

9b) Évaluation d'expressions

3 pts

Expression	Type C	Valeur
*p	int	-1 (p→0xA4 = 0b, *p = b = -1)
*t	char	'D' (0x44) (t→0xB8 = s[0])
strlen(s)	size_t	5 ("Dakar", 5 chars avant NUL)
strlen(t)	size_t	5 (t pointe vers s → même chaîne)
sizeof(s)	size_t	8 (s est char[8], taille du tableau)
sizeof(t)	size_t	8 (t est char*, pointeur 64 bits)
s[2]	char	'k' (0x6B) (3e octet de "Dakar")
t[3]	char	'a' (0x61) (t[3] = s[3] = 4e octet)
((short)(s + 1))	short	0x6B61 (à 0xB9 : octets 61, 6B → LE)
**pp	int	-1 (pp→0xA8=0p, *pp=p→0xA4=0b, **pp=b=-1)

Barème : A = 5 pts (Ex1 : 3, Ex2 : 2) B = 7 pts (Ex3 : 2, Ex4 : 2, Ex5 : 2, Ex6 : 1) C = 8 pts (Ex7 : 1, Ex8 : 3, Ex9 : 4)